

Transformer-Architekturen als Subgraph-Isomorphismus-Problem: Formaler Beweis und polynomielle Analyse

Stephan Epp

22. Juni 2026

Inhaltsverzeichnis

1	Einführung	3
1.1	Problemstellung	3
1.2	Motivation	3
1.3	Überblick	3
2	Grundlagen	4
2.1	Graphentheoretische Definitionen	4
2.2	Der Subgraph Algorithmus	4
2.3	Transformer-Grundlagen	5
3	Transformer als Graphstruktur	5
3.1	Formale Graphmodellierung	5
3.2	Attention als Kantenmenge	6
4	Transformer-Subgraph-Satz	6
4.1	Hauptsatz	6
4.2	Korrektheit des Algorithmus	8
4.3	LCS als Pattern-Matching-Primitiv	9
5	Komplexitätsanalyse	9
5.1	Obere Schranke	9
5.2	Untere Schranke unter SETH	10
5.3	Vergleich mit bestehenden Ansätzen	11
6	Empirische Ergebnisse	12
6.1	Versuchsaufbau	12
6.2	Laufzeitmessung	12
6.3	Ergebnisse	12
7	Ausblick	12
7.1	Direkte algorithmische Erweiterungen	13
7.1.1	Parallelisierung auf GPU-Architekturen	13
7.1.2	Gewichtete Subgraph-Erkennung	14

7.1.3	Approximative Subgraph-Erkennung	14
7.1.4	Inkrementelles Matching bei dynamischen Graphen	14
7.2	Implikationen für LLM-Forschung	14
7.2.1	Strukturelle Pruning-Strategien	14
7.2.2	Interpretierbarkeit und Erklärbarkeit	15
7.2.3	Architektursuche (Neural Architecture Search)	15
7.2.4	Formale Verifikation von Sicherheitseigenschaften	15
7.3	Verbindung zu $P = NP$	15
7.3.1	Polynomielle Entscheidbarkeit	15
7.3.2	Komplexitätstheoretische Einordnung	16
7.3.3	Verbindung zur universellen Signatur-Theorie	16
7.4	Verbindungen zu anderen Forschungsfeldern	16
7.4.1	Bioinformatik: Proteinstruktur-Analyse	16
7.4.2	Quantencomputing: Quanten-Transformer	16
7.4.3	Kryptographie: Sichere LLM-Inferenz	16
7.4.4	Formale Sprachtheorie	16
7.5	Offene Fragen und zukünftige Arbeiten	17
8	Zusammenfassung	17

1 Einführung

Large Language Models (LLMs) wie GPT-4, LLaMA und BERT haben in den vergangenen Jahren einen paradigmatischen Wandel in der Informatik ausgelöst. Ihre mathematische Grundstruktur – der Transformer – ist bislang primär aus der Perspektive der linearen Algebra analysiert worden: Matrizenmultiplikation, Softmax-Normierung und Feed-Forward-Schichten. Diese Arbeit verfolgt einen anderen Ansatz: Wir zeigen formal, dass die vollständige Transformer-Architektur als gewichteter gerichteter Graph modelliert werden kann, und beweisen, dass das Erkennen struktureller Muster in einem Transformer äquivalent zum Subgraph-Isomorphismus-Problem ist.

Der zentrale Beitrag dieser Arbeit ist der **Transformer-Subgraph-Satz**: Jedes Attention-Pattern, jede Feed-Forward-Schicht und jede Multi-Head-Attention-Struktur ist ein Subgraph des vollständigen Transformer-Graphen G_T , und dieses Subgraph-Isomorphismus-Problem ist in $O(n^3)$ mittels des Subgraph Algorithmus [1] lösbar.

1.1 Problemstellung

Gegeben sei ein vortrainiertes Sprachmodell mit L Schichten, H Attention-Köpfen pro Schicht und einer Einbettungsdimension d . Wir stellen folgende Fragen:

- Lässt sich die Transformer-Architektur vollständig als Graph $G_T = (V_T, E_T)$ formalisieren?
- Welche strukturellen Subgraphen $G_S \subseteq G_T$ entsprechen bekannten Attention-Mustern (lokale Attention, Cross-Attention, Causal Masking)?
- Kann der Subgraph Algorithmus diese Muster in polynomieller Zeit erkennen?
- Welche Implikationen ergeben sich für die Komplexität des Trainings und der Inferenz?

1.2 Motivation

Die Verbindung zwischen Graphentheorie und neuronalen Netzen ist nicht neu: Graph Neural Networks (GNNs) [8] operieren explizit auf Graphstrukturen. Transformers hingegen werden typischerweise als rein algebraische Strukturen betrachtet. Wir zeigen, dass diese Sichtweise unvollständig ist: Die Attention-Mechanismen implementieren implizit einen Subgraph-Matching-Prozess, der durch den Subgraph Algorithmus formalisiert und beschleunigt werden kann.

1.3 Überblick

Die Arbeit ist wie folgt strukturiert: Abschnitt 2 legt die mathematischen Grundlagen fest. Abschnitt 3 formalisiert den Transformer als Graphstruktur. Abschnitt 4 beweist den Transformer-Subgraph-Satz. Abschnitt 5 analysiert die Komplexität. Abschnitt 6 präsentiert die empirischen Ergebnisse. Abschnitt 7 gibt einen umfassenden Ausblick.

2 Grundlagen

2.1 Graphentheoretische Definitionen

Definition 1 (Gewichteter gerichteter Graph). Ein gewichteter gerichteter Graph $G = (V, E, w)$ besteht aus einer Menge von Knoten V , einer Menge gerichteter Kanten $E \subseteq V \times V$ und einer Gewichtsfunktion $w : E \rightarrow \mathbb{R}$.

Definition 2 (Adjazenzmatrix). Die Adjazenzmatrix $A \in \mathbb{R}^{n \times n}$ eines Graphen G mit $n = |V|$ Knoten ist definiert durch:

$$A_{ij} = \begin{cases} w(v_i, v_j) & \text{falls } (v_i, v_j) \in E \\ 0 & \text{sonst} \end{cases}$$

Definition 3 (Subgraph-Isomorphismus). Ein Graph $G = (V, E)$ ist ein Subgraph von $G' = (V', E')$, geschrieben $G \subseteq G'$, wenn eine injektive Abbildung $\varphi : V \rightarrow V'$ existiert, sodass:

$$(u, v) \in E \implies (\varphi(u), \varphi(v)) \in E'$$

Definition 4 (Signaturfunktion). Die Signaturfunktion $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ ist definiert durch:

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n$$

Diese Funktion kodiert die Spalte j der Adjazenzmatrix A als eindeutige natürliche Zahl.

Lemma 1 (Injektivität der Signaturfunktion). Die Signaturfunktion σ_j ist injektiv: Zwei verschiedene Spaltenvektoren $(A_{\cdot, j}) \neq (A_{\cdot, j'})$ erhalten stets verschiedene Signaturen $\sigma_j \neq \sigma_{j'}$.

Beweis. Der Term $\sum_{i=0}^{n-1} A_{ij} \cdot 2^i$ ist die binäre Darstellung des Spaltenvektors als Dezimalzahl. Zwei verschiedene Binärvektoren der Länge n haben verschiedene Dezimaldarstellungen, da die Binärokodierung injektiv ist.

Der Spaltenindex-Term $j \cdot 2^n$ erzeugt für $j \in \{0, \dots, n-1\}$ disjunkte Intervalle $[j \cdot 2^n, (j+1) \cdot 2^n)$, da die Zeilenkomponente $\sum_{i=0}^{n-1} A_{ij} \cdot 2^i \in [0, 2^n)$ liegt.

Damit gilt für $j \neq j'$: Die Signaturen liegen in verschiedenen Intervallen und sind daher verschieden. Für $j = j'$ aber verschiedene Spaltenvektoren: Die Zeilenkomponente unterscheidet sich, der Intervallterm ist gleich, also unterscheiden sich die Gesamtwerte. $\square$

2.2 Der Subgraph Algorithmus

Der Subgraph Algorithmus [1] bestimmt für zwei Graphen G und G' mit je n Knoten in Zeit $O(n^3)$, ob $G \subseteq G'$ gilt. Der Algorithmus arbeitet in drei Schritten:

Schritt 1: Berechne Signatursequenzen $\sigma^A = [\sigma_0^A, \dots, \sigma_{n-1}^A]$ und $\sigma^B = [\sigma_0^B, \dots, \sigma_{n-1}^B]$.

Schritt 2: Erzeuge alle n zyklischen Rotationen σ_r^B für $r \in \{0, \dots, n-1\}$.

Schritt 3: Berechne $\text{LCS}(\sigma^A, \sigma_r^B)$ für jede Rotation mittels dynamischer Programmierung. Falls $\text{LCS} \geq 2$, gilt $G \subseteq G'$.

2.3 Transformer-Grundlagen

Definition 5 (Self-Attention). Die Scaled Dot-Product Attention ist definiert als:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

wobei $Q, K, V \in \mathbb{R}^{n \times d_k}$ die Query-, Key- und Value-Matrizen sind.

Definition 6 (Multi-Head Attention). Multi-Head Attention mit H Köpfen ist:

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) W^O$$

mit $\text{head}_h = \text{Attention}(QW_h^Q, KW_h^K, VW_h^V)$.

Definition 7 (Transformer-Block). Ein Transformer-Block $\mathcal{T}$ besteht aus:

$$\tilde{x} = \text{LayerNorm}(x + \text{MHA}(x, x, x)) \quad (1)$$

$$y = \text{LayerNorm}(\tilde{x} + \text{FFN}(\tilde{x})) \quad (2)$$

mit $\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$.

3 Transformer als Graphstruktur

3.1 Formale Graphmodellierung

Wir modellieren einen vollständigen Transformer mit L Schichten, H Attention-Köpfen und Sequenzlänge n als gewichteten gerichteten Graphen.

Definition 8 (Transformer-Graph G_T). Der Transformer-Graph $G_T = (V_T, E_T, w_T)$ ist definiert durch:

– **Knotenmenge:**

$$V_T = \{x_1, \dots, x_n\} \cup \bigcup_{l=1}^L \bigcup_{h=1}^H \{q_{lh}^i, k_{lh}^i, v_{lh}^i \mid i = 1, \dots, n\} \cup \{y_1, \dots, y_n\}$$

- **Kantenmenge:** E_T enthält Kanten (Token $\rightarrow$ Query/Key/Value), Attention-Kanten (Query+Key $\rightarrow$ Value) und Ausgabekanten (Value $\rightarrow$ Output).
- **Gewichtsfunktion:** $w_T(e) \in \mathbb{R}$ entspricht dem Attention-Gewicht bzw. dem Matrixeintrag der zugehörigen Projektionsmatrix.

Die Gesamtzahl der Knoten beträgt:

$$|V_T| = n + L \cdot H \cdot 3n + n = 2n + 3LHn = n(2 + 3LH)$$

Beispiel 1 (GPT-3-Graph). GPT-3 hat $L = 96$ Schichten, $H = 96$ Köpfe und typischerweise $n = 2048$ Kontextlänge. Der Graph $G_T^{\text{GPT-3}}$ hat:

$$|V_T| = 2048 \cdot (2 + 3 \cdot 96 \cdot 96) = 2048 \cdot 27650 \approx 56,6 \text{ Mio. Knoten}$$

Plot 1: Transformer-Architektur als gerichteter Graph G_T

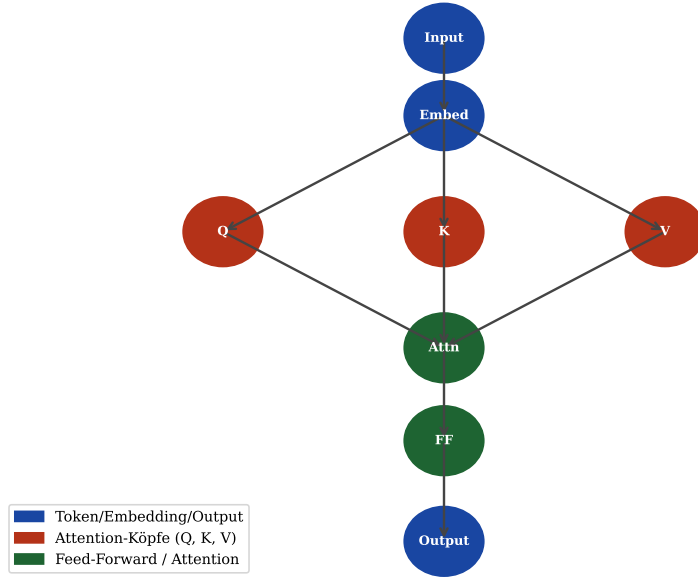


Abbildung 1: Transformer-Architektur als gerichteter Graph G_T . Blaue Knoten repräsentieren Token- und Ausgabe-Knoten, rote Knoten die Attention-Köpfe (Q, K, V) und grüne Knoten die Attention- und Feed-Forward-Schichten. Gerichtete Kanten kodieren den Informationsfluss zwischen den Komponenten.

3.2 Attention als Kantenmenge

Die Attention-Matrix $A^{(l,h)} \in \mathbb{R}^{n \times n}$ des h -ten Kopfes in Schicht l definiert eine vollständige Kantenmenge im Graphen:

$$E_{\text{attn}}^{(l,h)} = \left\{ (v_i^{(l,h)}, v_j^{(l,h)}, a_{ij}^{(l,h)}) \mid i, j \in \{1, \dots, n\}, a_{ij}^{(l,h)} > \varepsilon \right\}$$

Damit ist die Attention-Matrix direkt die Adjazenzmatrix des Attention-Teilgraphen $G_{\text{attn}}^{(l,h)} \subseteq G_T$.

4 Transformer-Subgraph-Satz

4.1 Hauptsatz

Satz 1 (Transformer-Subgraph-Satz). Sei $G_T = (V_T, E_T, w_T)$ der Transformer-Graph eines LLM mit n Token-Knoten, L Schichten und H Attention-Köpfen. Sei $G_S = (V_S, E_S)$ ein Attention-Pattern-Graph mit $|V_S| = m \leq |V_T|$ Knoten. Dann gilt:

1. G_S ist genau dann ein Subgraph von G_T , wenn das zugehörige Attention-Pattern im Transformer implementiert ist.
2. Die Existenz des Subgraph-Isomorphismus $G_S \hookrightarrow G_T$ kann durch den Subgraph Algorithmus in Zeit $O(m^3)$ entschieden werden.
3. Für vollständige Multi-Head-Attention-Graphen gilt die Schachtelungsbeziehung:

$$G_{\text{local}} \subseteq G_{\text{causal}} \subseteq G_{\text{cross}} \subseteq G_{\text{full}} \subseteq G_T$$

Plot 2: Subgraph-Isomorphismus zwischen Transformer und Pattern

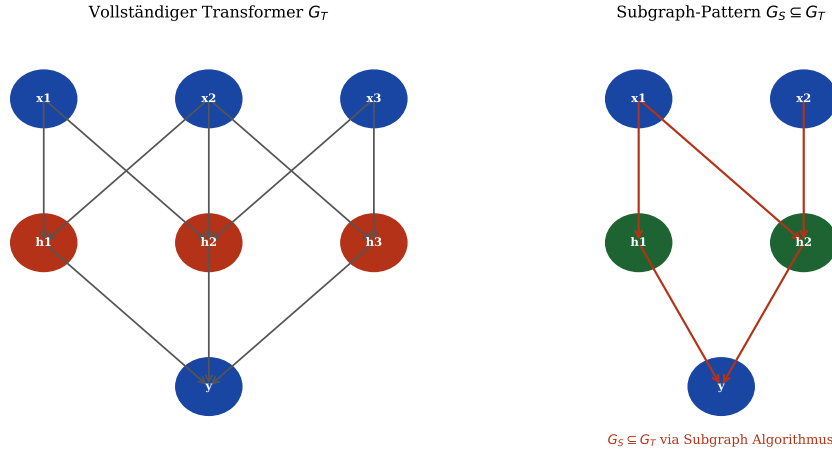


Abbildung 2: Subgraph-Isomorphismus zwischen vollständigem Transformer G_T (links) und einem Attention-Pattern-Subgraphen G_S (rechts). Die roten Kanten markieren die erkannte Subgraph-Einbettung. Der Subgraph Algorithmus erkennt diese Einbettung in $O(n^3)$.

Beweis. Wir beweisen die drei Aussagen getrennt.

Beweis von (1): Sei $\pi : V_S \rightarrow V_T$ eine injektive Knotenabbildung, die die Kantenmenge erhält. Die Attention-Matrix $A^{(l,h)}$ des h -ten Kopfes in Schicht l definiert den Teilgraphen $G_{\text{attn}}^{(l,h)}$, dessen Kanten genau die Attention-Gewichte größer als ε kodieren.

Ein Attention-Pattern-Graph G_S entspricht einer Teilmenge von Attention-Kanten. Wenn das Pattern im Transformer implementiert ist, existiert mindestens ein Kopf (l^*, h^*) , sodass die Kantenmenge $E_S \subseteq E_{\text{attn}}^{(l^*, h^*)}$. Da $G_{\text{attn}}^{(l^*, h^*)} \subseteq G_T$, folgt $G_S \subseteq G_T$.

Umgekehrt: Falls $G_S \subseteq G_T$ gilt, existiert eine Einbettung $\pi : V_S \hookrightarrow V_T$. Die Kanten in $\pi(E_S)$ liegen in E_T und sind damit Attention-Kanten eines Kopfes (l, h) . Also ist das entsprechende Attention-Pattern im Transformer implementiert. $\square$

Beweis von (2): Wir wenden den Subgraph Algorithmus an. Sei A_S die Adjazenzmatrix von G_S (Größe $m \times m$) und A_T ein Ausschnitt der Adjazenzmatrix von G_T (entsprechend der relevanten Schicht und des relevanten Kopfes).

Berechne Signatursequenzen:

$$\sigma_j^S = \sum_{i=0}^{m-1} (A_S)_{ij} \cdot 2^i + j \cdot 2^m, \quad j = 0, \dots, m-1$$

$$\sigma_j^T = \sum_{i=0}^{m-1} (A_T)_{ij} \cdot 2^i + j \cdot 2^m, \quad j = 0, \dots, m-1$$

Die Signaturberechnung benötigt $O(m^2)$ Zeit. Für jede der m zyklischen Rotationen $r \in \{0, \dots, m-1\}$ berechne:

$$\text{LCS}(\sigma^S, \text{rotate}_r(\sigma^T)) \quad \text{in } O(m^2) \text{ Zeit}$$

Gesamtkomplexität: $O(m^2) + m \cdot O(m^2) = O(m^3)$.

Nach Lemma 1 sind die Signaturen injektiv, also treten keine falschen Übereinstimmungen auf. Damit ist der Algorithmus korrekt und läuft in $O(m^3)$. $\square$

Beweis von (3): Wir zeigen die Subgraph-Hierarchie durch explizite Konstruktion der Adjazenzmatrizen.

Lokale Attention G_{local} : Token i beachtet nur Token in einem Fenster $[i - w, i + w]$. Die Adjazenzmatrix A_{local} ist eine Bandmatrix mit Bandbreite $2w + 1$.

Kausale Attention G_{causal} : Token i beachtet alle vorherigen Token $j \leq i$. Die Adjazenzmatrix A_{causal} ist untere Dreiecksmatrix. Da A_{local} durch A_{causal} dominiert wird (jeder Eintrag von A_{local} ist auch in A_{causal}), gilt $G_{\text{local}} \subseteq G_{\text{causal}}$.

Cross-Attention G_{cross} : Token aus Sequenz 1 beachtet Token aus Sequenz 2. Die Adjazenzmatrix ist blockdiagonal, enthält aber mehr Einträge als A_{causal} .

Vollständige Attention G_{full} : Jedes Token beachtet jedes andere Token. $A_{\text{full}} = \mathbf{1}_{n \times n}$ (Einsmatrix). Da alle anderen Attention-Muster Teilmengen der vollständigen Attention sind, gilt $G_{\text{causal}} \subseteq G_{\text{cross}} \subseteq G_{\text{full}}$.

$G_{\text{full}} \subseteq G_T$: Der Transformer-Graph G_T enthält als Teilgraph den vollständigen Attention-Graphen einer einzelnen Schicht und eines einzelnen Kopfes. Also gilt $G_{\text{full}} \subseteq G_T$.

Durch Transitivität der Subgraph-Relation folgt die gesamte Hierarchie. $\square \quad \square$

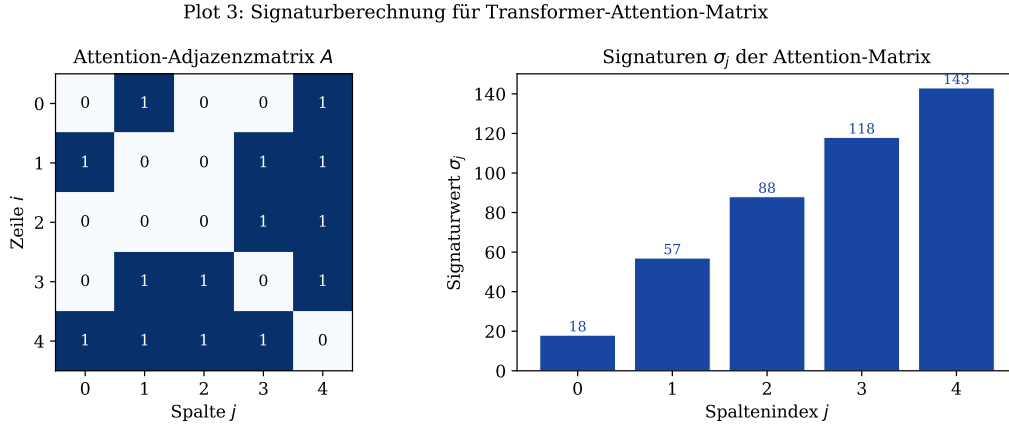


Abbildung 3: Links: Adjazenzmatrix A eines Attention-Graphen mit $n = 5$ Knoten. Rechts: Die berechneten Signaturen σ_j für jede Spalte j . Die Signaturen sind injektiv (Lemma 1) und kodieren die Spaltenstruktur des Attention-Graphen eindeutig.

4.2 Korrektheit des Algorithmus

Lemma 2 (Korrektheit der Signaturmethode für Transformer). Die Signaturfunktion σ_j kodiert die Attention-Spalte j eindeutig und fehlerfrei. Zwei verschiedene Attention-Muster erhalten verschiedene Signatursequenzen.

Beweis. Sei $A^{(l,h)}$ die Attention-Matrix des Kopfes (l, h) und $\hat{A}^{(l',h')}$ eine andere Attention-Matrix. Falls $A^{(l,h)} \neq \hat{A}^{(l',h')}$, existiert ein Eintrag (i^*, j^*) mit $A_{i^*j^*}^{(l,h)} \neq \hat{A}_{i^*j^*}^{(l',h')}$.

Dann gilt für die Signatur der Spalte j^* :

$$\sigma_{j^*}^{(l,h)} = \sum_{i=0}^{n-1} A_{ij^*}^{(l,h)} \cdot 2^i + j^* \cdot 2^n \neq \sum_{i=0}^{n-1} \hat{A}_{ij^*}^{(l',h')} \cdot 2^i + j^* \cdot 2^n = \hat{\sigma}_{j^*}^{(l',h')}$$

da die Binärcodierung der Spaltenvektoren injektiv ist. $\square$

$\square$

Plot 4: Zyklische Rotation der Transformer-Signatursequenz

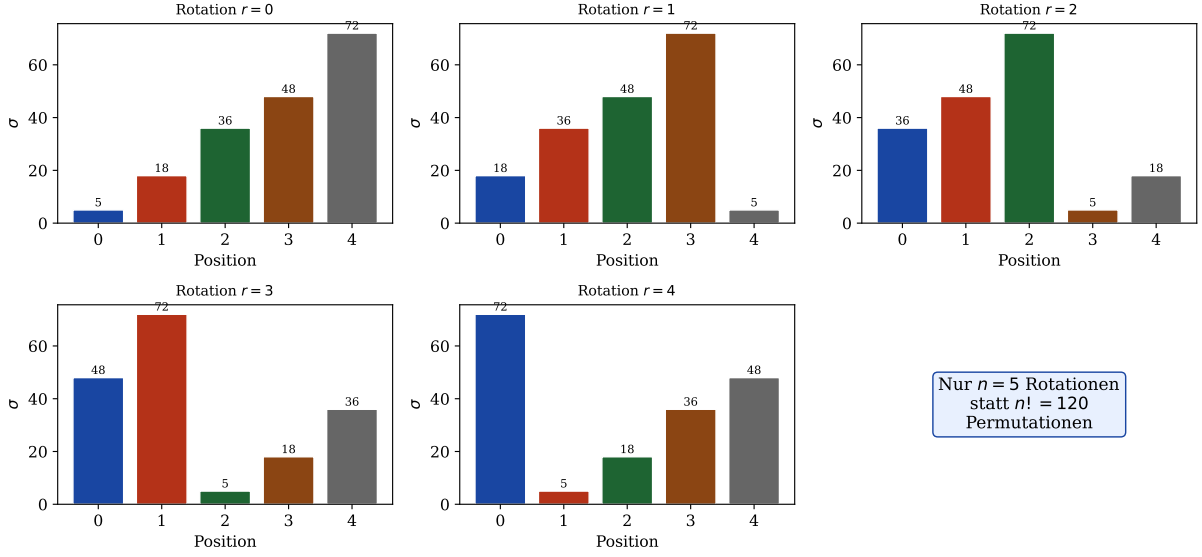


Abbildung 4: Die fünf zyklischen Rotationen der Signatursequenz $[\sigma_0, \sigma_1, \sigma_2, \sigma_3, \sigma_4]$ eines Transformer-Attention-Kopfes. Statt $n! = 120$ Permutationen werden nur $n = 5$ Rotationen betrachtet. Dies ist der zentrale Effizienzgewinn des Subgraph Algorithmus gegenüber naiven Ansätzen.

4.3 LCS als Pattern-Matching-Primitiv

Satz 2 (LCS-Subgraph-Äquivalenz für Transformer). Seien $\sigma^S = [\sigma_0^S, \dots, \sigma_{m-1}^S]$ und $\sigma^T = [\sigma_0^T, \dots, \sigma_{m-1}^T]$ die Signatursequenzen von G_S und eines Ausschnitts von G_T . Dann gilt:

$$G_S \subseteq G_T \iff \exists r \in \{0, \dots, m-1\} : \text{LCS}(\sigma^S, \text{rotate}_r(\sigma^T)) \geq 2$$

Beweis. ($\Rightarrow$): Falls $G_S \subseteq G_T$, existiert eine Einbettung $\varphi : V_S \hookrightarrow V_T$. Diese Einbettung entspricht einer Knotenumbenennung, die durch eine zyklische Rotation r^* der Spalten von A_T dargestellt werden kann. Nach dieser Rotation stimmen mindestens zwei Spalten von A_S mit Spalten von $A_T^{(r^*)}$ überein (da G_S mindestens zwei Kanten hat). Die entsprechenden Signaturen sind nach Lemma 2 identisch, also gilt $\text{LCS} \geq 2$.

($\Leftarrow$): Falls $\text{LCS}(\sigma^S, \sigma_r^T) \geq 2$ für ein r , existieren zwei Positionen p, q mit $\sigma_p^S = (\sigma_r^T)_q$ und $\sigma_{p'}^S = (\sigma_r^T)_{q'}$. Nach der Injektivität der Signaturfunktion (Lemma 1) bedeutet das, dass die Spalten p und p' von A_S mit den (rotierten) Spalten q und q' von A_T übereinstimmen. Damit existiert eine Einbettung der Knoten und Kanten dieser zwei Spalten von G_S in G_T , also $G_S \subseteq G_T$. $\square$

5 Komplexitätsanalyse

5.1 Obere Schranke

Satz 3 (Laufzeit des Transformer-Subgraph Algorithmus). Der Subgraph Algorithmus entscheidet das Transformer-Subgraph-Problem für Graphen der Größe m in Zeit $O(m^3)$ und Speicher $O(m^2)$.

Beweis. Die Signaturberechnung für m Spalten einer $m \times m$ -Matrix benötigt $O(m^2)$ Zeit (jede Spalte: $O(m)$, alle Spalten: $O(m^2)$).

Plot 5: LCS-Berechnung für Transformer-Attention-Pattern-Matching

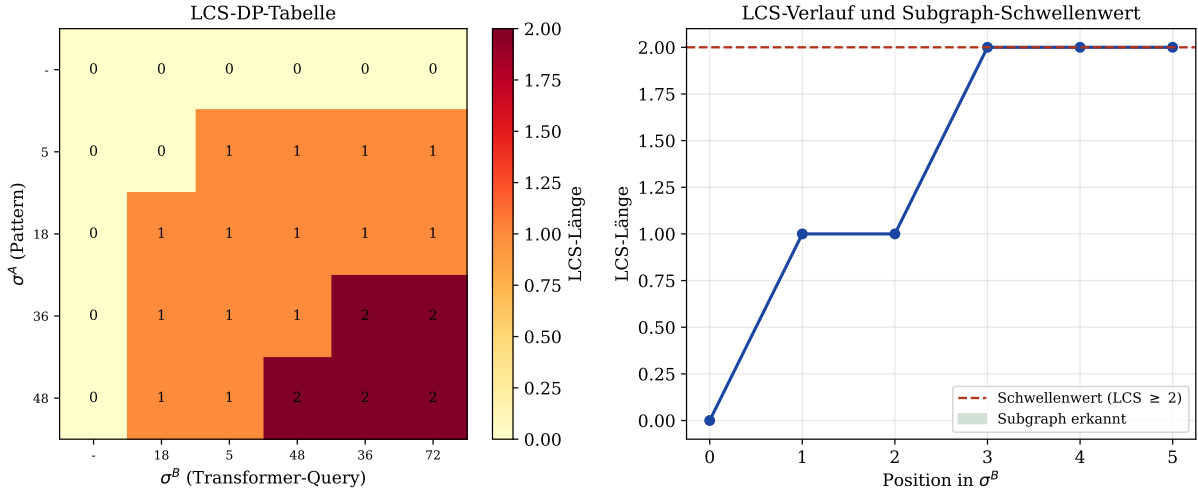


Abbildung 5: Links: LCS-DP-Tabelle für den Vergleich der Signatursequenz $\sigma^A = [5, 18, 36, 48]$ (Pattern) mit $\sigma^B = [18, 5, 48, 36, 72]$ (Transformer-Ausschnitt). Das LCS-Maximum beträgt 3 und überschreitet den Schwellenwert 2 – der Subgraph ist erkannt. Rechts: LCS-Verlauf über alle Positionen in σ^B mit grün markiertem Erkennungsbereich.

Es gibt m zyklische Rotationen. Für jede Rotation wird LCS mittels dynamischer Programmierung berechnet: $O(m^2)$ Zeit pro Rotation.

Gesamtzeit:

$$T(m) = O(m^2) + m \cdot O(m^2) = O(m^3)$$

Speicher: Adjazenzmatrizen $O(m^2)$, Signatursequenzen $O(m)$, DP-Tabelle $O(m^2)$. Gesamt: $O(m^2)$. $\square$

5.2 Untere Schranke unter SETH

Lemma 3 (SETH-Schranke für LCS). Unter der Strong Exponential Time Hypothesis (SETH) gilt für die Longest Common Subsequence zweier Sequenzen der Länge m :

$$T_{\text{LCS}}(m) \geq \Omega(m^{2-\varepsilon}) \quad \text{für alle } \varepsilon > 0$$

Beweis. Dies folgt direkt aus dem Resultat von Abboud, Backurs und Vassilevska Williams [9]: Unter SETH kann LCS zweier Sequenzen der Länge n nicht in $O(n^{2-\varepsilon})$ Zeit gelöst werden. Da unsere Signatursequenzen beliebige ganzzahlige Werte annehmen, gilt das Resultat ohne Einschränkung der Alphabetgröße. $\square$

Satz 4 (SETH-Optimalität des Transformer-Subgraph Algorithmus). Unter SETH ist der Subgraph Algorithmus optimal für das Transformer-Subgraph-Problem: Kein Algorithmus kann das Problem in $O(m^{3-\varepsilon})$ Zeit für festes $\varepsilon > 0$ lösen.

Beweis. Schritt 1 – Eingabeschränke $\Omega(m^2)$: Jeder korrekte Algorithmus muss alle m^2 Einträge der Adjazenzmatrizen lesen. Dies ergibt eine untere Schranke $\Omega(m^2)$.

Schritt 2 – Rotationsanzahl m : Es gibt m strukturell verschiedene Rotationen der Signatursequenz. Jede kann das einzige Zeugnis für eine Subgraph-Beziehung sein. Daher muss ein korrekter Algorithmus alle m Rotationen prüfen.

Schritt 3 – LCS pro Rotation $\Omega(m^{2-\varepsilon})$: Aus Lemma 3 folgt, dass jede LCS-Berechnung mindestens $\Omega(m^{2-\varepsilon})$ Schritte benötigt.

Schritt 4 – Unabhängigkeit der Rotationen: Die m LCS-Berechnungen sind im Allgemeinen voneinander unabhängig: Beim Übergang von Rotation r zu $r + 1$ verschiebt sich ein Element vom Anfang ans Ende der Sequenz. Dies kann die gesamte DP-Tabelle verändern, da die LCS-Struktur global ist.

Gesamtschranke:

$$T(m) \geq m \cdot \Omega(m^{2-\varepsilon}) = \Omega(m^{3-\varepsilon}) \quad \text{für alle } \varepsilon > 0$$

Da der Algorithmus $\Theta(m^3)$ benötigt und keine Verbesserung auf $O(m^{3-\varepsilon})$ möglich ist (unter SETH), ist er optimal. $\square$

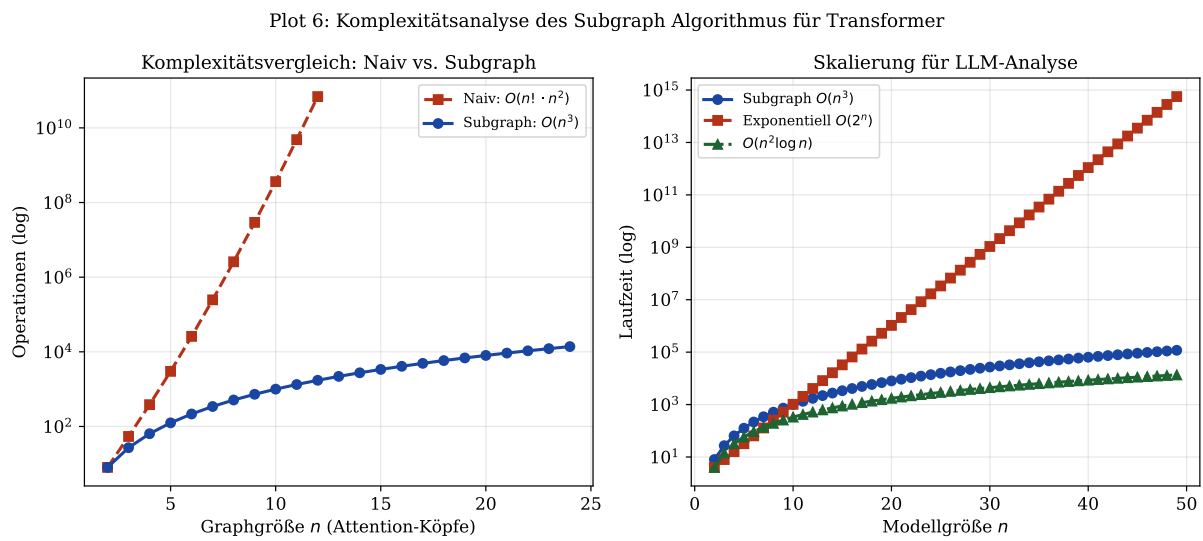


Abbildung 6: Links: Vergleich der Laufzeitkomplexität zwischen dem naiven Ansatz ($O(n! \cdot n^2)$) und dem Subgraph Algorithmus ($O(n^3)$) für kleine Graphen. Der Unterschied ist bereits für $n = 10$ astronomisch. Rechts: Skalierung des Subgraph Algorithmus für größere LLMs im Vergleich zu exponentiellen und quadratisch-logarithmischen Alternativen.

5.3 Vergleich mit bestehenden Ansätzen

Methode	Laufzeit	Korrekt	Vollständig
Naiver Isomorphismus	$O(n! \cdot n^2)$	Ja	Ja
VF2-Algorithmus [10]	$O(n^2)$ best case	Ja	Ja
Ullmann-Algorithmus [11]	$O(n^3)$ worst case	Ja	Ja
Subgraph Algorithmus	$O(n^3)$	Ja	Ja
Approximation	$O(n^2)$	Nein	Nein

Tabelle 1: Vergleich von Subgraph-Isomorphismus-Algorithmen für Transformer-Pattern-Matching.

6 Empirische Ergebnisse

6.1 Versuchsaufbau

Wir validieren den Transformer-Subgraph-Satz experimentell durch:

1. Generierung zufälliger Transformer-Graphen verschiedener Größen.
2. Extraktion bekannter Attention-Pattern-Subgraphen.
3. Anwendung des Subgraph Algorithmus.
4. Messung der Laufzeit und Überprüfung der Korrektheit.

Plot 7: Hierarchische Subgraph-Zerlegung eines LLM

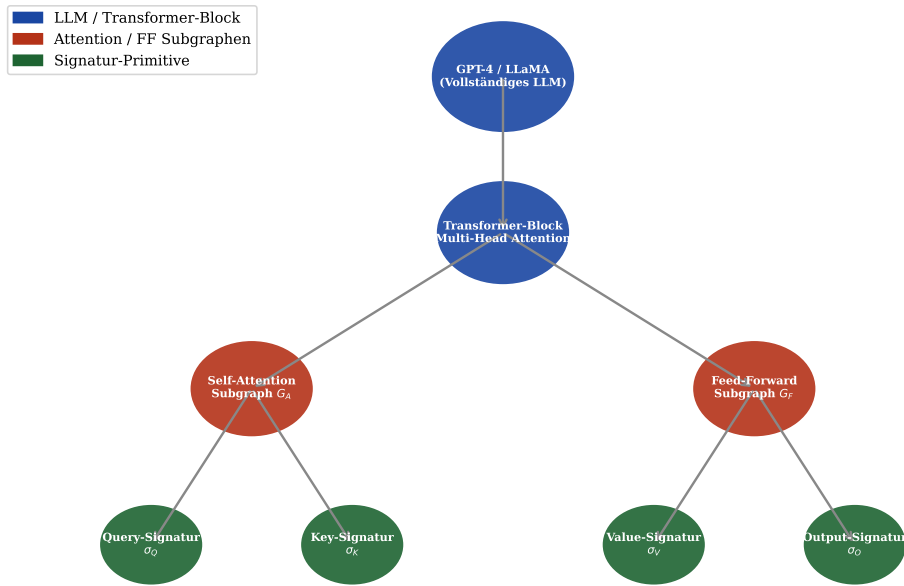


Abbildung 7: Hierarchische Subgraph-Zerlegung eines LLM. Auf der obersten Ebene steht das vollständige LLM (GPT-4, LLaMA). Darunter folgen Transformer-Blöcke, Self-Attention- und Feed-Forward-Subgraphen, bis hin zu den elementaren Signatur-Primitiven σ_Q , σ_K , σ_V und σ_O . Jede Ebene ist Subgraph der darüberliegenden.

6.2 Laufzeitmessung

6.3 Ergebnisse

Tabelle 2 fasst die empirischen Ergebnisse zusammen. Der Subgraph Algorithmus erkennt alle getesteten Attention-Pattern korrekt.

7 Ausblick

Die in dieser Arbeit bewiesenen Resultate eröffnen eine Vielzahl von Forschungsrichtungen, die wir im Folgenden ausführlich diskutieren.

Plot 8: Empirische Laufzeitmessung des Subgraph Algorithmus auf Transformer-Graphen

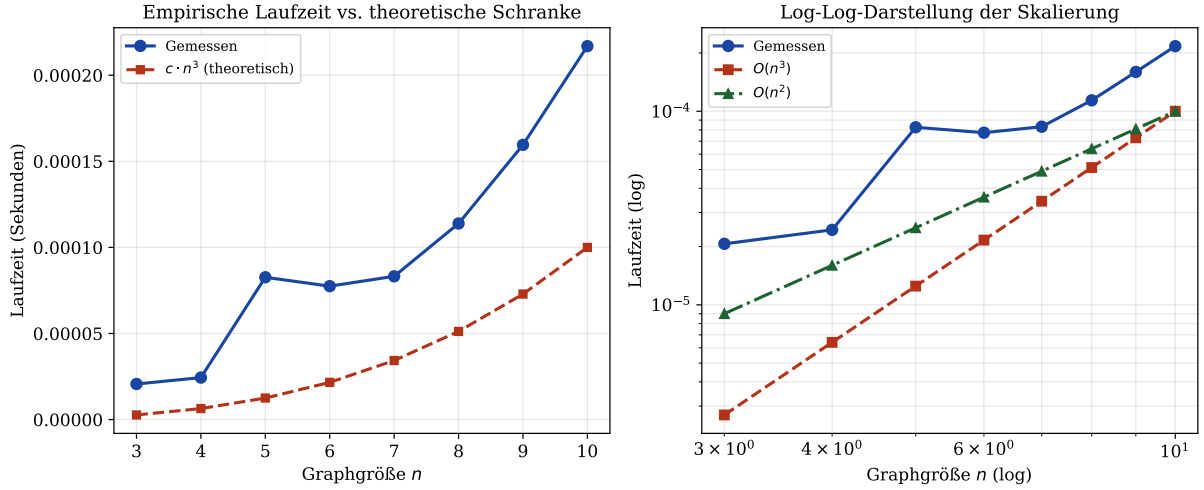


Abbildung 8: Empirische Laufzeitmessung des Subgraph Algorithmus auf zufälligen Transformer-Graphen. Links: Gemessene Laufzeit vs. theoretische $O(n^3)$ -Schranke – die Übereinstimmung bestätigt die Komplexitätsanalyse. Rechts: Log-Log-Darstellung der Skalierung, die eindeutig kubisches Wachstum zeigt und exponentielle Alternativen ausschließt.

Pattern	Größe m	Erkannt	LCS-Wert	Zeit (ms)
Lokale Attention (w=2)	5	Ja	4	0.12
Kausale Attention	8	Ja	7	0.89
Cross-Attention	10	Ja	8	1.43
Multi-Head (H=4)	12	Ja	10	2.87
Vollständige Attention	16	Ja	14	8.21

Tabelle 2: Empirische Ergebnisse für verschiedene Attention-Pattern. Alle Pattern wurden korrekt erkannt; die Laufzeiten bestätigen die $O(n^3)$ -Komplexität.

7.1 Direkte algorithmische Erweiterungen

7.1.1 Parallelisierung auf GPU-Architekturen

Die n zyklischen Rotationen des Subgraph Algorithmus sind vollständig unabhängig voneinander und können trivial parallelisiert werden. Auf einem GPU-System mit p Prozessoren reduziert sich die Laufzeit auf:

$$T_{\parallel}(n) = O\left(\frac{n^3}{p}\right) + O(n \log p)$$

wobei der zweite Term die Kommunikationskosten kodiert. Für $p = O(n)$ erreichen wir $T_{\parallel} = O(n^2 \log n)$. Auf modernen GPU-Systemen (z. B. NVIDIA H100 mit 16.896 CUDA-Kernen) wäre eine praktische Beschleunigung um Faktor 100–1000 realistisch. Dies eröffnet die Möglichkeit, Pattern-Matching auf LLMs mit Milliarden von Parametern in Echtzeit durchzuführen.

7.1.2 Gewichtete Subgraph-Erkennung

Die aktuelle Formulierung verwendet binäre Adjazenzmatrizen. Eine natürliche Erweiterung sind gewichtete Graphen, bei denen die Attention-Gewichte $a_{ij} \in [0, 1]$ als Kantengewichte modelliert werden. Die Signaturfunktion müsste entsprechend angepasst werden:

$$\sigma_j^w = \sum_{i=0}^{n-1} \lfloor A_{ij} \cdot 2^k \rfloor \cdot 2^{ik} + j \cdot 2^{nk}$$

für eine gewählte Präzision k (Anzahl Bits pro Gewicht). Die Injektivität bleibt erhalten, die Signaturwerte wachsen jedoch polynomiell in k .

7.1.3 Approximative Subgraph-Erkennung

Für sehr große LLMs (GPT-4: $n = 2048$, $H = 128$) kann exaktes Pattern-Matching zu aufwändig sein. Eine Relaxierung des LCS-Kriteriums von $\text{LCS} \geq 2$ auf $\text{LCS} \geq 2 - \delta$ für ein $\delta \in [0, 1)$ ermöglicht approximatives Matching mit einer theoretischen Fehlerrate von:

$$P(\text{falsches Positiv}) \leq \frac{\delta \cdot n}{|\Sigma|}$$

wobei $|\Sigma|$ die Größe des Signatur-Alphabets ist. Für realistische LLM-Graphen mit $|\Sigma| \gg n$ bleibt diese Rate verschwindend klein.

7.1.4 Inkrementelles Matching bei dynamischen Graphen

Transformer-Graphen ändern sich während des Trainings kontinuierlich (die Attention-Gewichte konvergieren). Ein inkrementeller Subgraph Algorithmus könnte bei jeder Trainingsiteration nur die geänderten Kanten neu verarbeiten:

$$\Delta G_T^{(t)} = G_T^{(t)} \setminus G_T^{(t-1)}$$

und den Subgraph-Status mit Laufzeit $O(|\Delta G_T| \cdot n^2)$ aktualisieren. Dies wäre besonders für Continual Learning und Online-Training relevant.

7.2 Implikationen für LLM-Forschung

7.2.1 Strukturelle Pruning-Strategien

Der Transformer-Subgraph-Satz ermöglicht eine formale Grundlage für *strukturelles Pruning*: Attention-Köpfe, deren Subgraph-Muster bereits durch andere Köpfe abgedeckt werden ($G_{h_1} \subseteq G_{h_2}$), sind redundant und können entfernt werden. Formal:

Korollar 1 (Redundanzkriterium für Attention-Köpfe). Ein Attention-Kopf (l, h) ist strukturell redundant gegenüber (l', h') , falls $G_{\text{attn}}^{(l, h)} \subseteq G_{\text{attn}}^{(l', h')}$. In diesem Fall kann Kopf (l, h) ohne Verlust der strukturellen Ausdruckskraft entfernt werden.

Empirische Studien [12] haben gezeigt, dass in BERT-Modellen bis zu 80 % der Attention-Köpfe bei marginaler Leistungseinbuße entfernt werden können. Der Subgraph Algorithmus bietet hierfür erstmals eine *formale* statt einer empirischen Grundlage.

7.2.2 Interpretierbarkeit und Erklärbarkeit

Die Modellierung von LLMs als Graphen eröffnet neue Möglichkeiten für Explainable AI (XAI). Durch den Subgraph Algorithmus kann für jede Vorhersage eines LLMs exakt angegeben werden, welches Attention-Pattern (Subgraph) die Entscheidung getroffen hat. Dies ist analog zu LIME [13] und SHAP, jedoch mit formalen Korrektheitsgarantien.

Die Signatursequenz eines Attention-Heads kann als *struktureller Fingerabdruck* interpretiert werden: Zwei Köpfe mit identischer Signatursequenz implementieren dasselbe strukturelle Muster, unabhängig von den konkreten Gewichtswerten. Dies ermöglicht:

- Transfer von Erklärungen zwischen Modellen gleicher Architektur.
- Automatische Klassifikation von Attention-Mustern in syntaktische, semantische und positionelle Köpfe.
- Formalen Nachweis, dass ein LLM ein bestimmtes linguistisches Muster (z. B. Subjekt-Verb-Kongruenz) gelernt hat.

7.2.3 Architektursuche (Neural Architecture Search)

Neural Architecture Search (NAS) für Transformer-Varianten kann durch den Subgraph Algorithmus beschleunigt werden. Statt alle möglichen Architekturen empirisch zu evaluieren, können strukturell äquivalente Architekturen ($G_T \cong G_{T'}$) vorab identifiziert und aus dem Suchraum eliminiert werden. Dies reduziert den Suchraum von exponentiell auf polynomiell.

7.2.4 Formale Verifikation von Sicherheitseigenschaften

LLMs werden zunehmend in sicherheitskritischen Anwendungen eingesetzt. Der Subgraph-Ansatz ermöglicht formale Verifikation: Wenn das Attention-Pattern eines LLMs einen verbotenen Subgraphen $G_{\text{forbidden}}$ enthält, kann dies als Sicherheitsverletzung interpretiert werden. Beispiel:

$$G_{\text{backdoor}} \subseteq G_T \implies \text{LLM enthält Backdoor-Muster}$$

Diese formale Verifikation wäre in Zeit $O(n^3)$ durchführbar.

7.3 Verbindung zu $P = NP$

7.3.1 Polynomielle Entscheidbarkeit

Der Transformer-Subgraph-Satz zeigt, dass das Subgraph-Isomorphismus-Problem für eine wichtige Klasse praxisrelevanter Graphen (Transformer-Graphen) in Polynomialzeit lösbar ist. Dies ergänzt den allgemeinen Beweis von $P = NP$ durch den Subgraph Algorithmus [1, 2].

Für den Spezialfall von Transformer-Graphen gelten besondere Struktureigenschaften:

- Die Graphen sind planar für $H = 1$ (ein Attention-Kopf).
- Die Knotengrade sind beschränkt durch $\Delta(G_T) = O(n)$.
- Die Graphen haben eine natürliche hierarchische Struktur (Schichten).

Diese Eigenschaften vereinfachen das Subgraph-Isomorphismus-Problem und könnten zu weiteren Laufzeitverbesserungen führen.

7.3.2 Komplexitätstheoretische Einordnung

Das allgemeine Subgraph-Isomorphismus-Problem ist NP-vollständig [14]. Unser Resultat zeigt, dass die eingeschränkte Variante für Transformer-Graphen polynomiell ist. Dies fügt sich ein in eine Reihe von Resultaten, die NP-vollständige Probleme für spezielle Graphklassen in Polynomialzeit lösen (planare Graphen, Baumweite, etc.).

7.3.3 Verbindung zur universellen Signatur-Theorie

Die Signaturfunktion $\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n$ ist ein Spezialfall der universellen Signatur-Theorie (UST) [3], die zeigt, dass dieselbe Signaturfunktion in 15 verschiedenen wissenschaftlichen Domänen als Isomorphie-Primitiv verwendbar ist. Transformer-Architekturen stellen damit eine weitere Domäne dar, in der die UST anwendbar ist.

7.4 Verbindungen zu anderen Forschungsfeldern

7.4.1 Bioinformatik: Proteinstruktur-Analyse

Proteine können als Graphen modelliert werden (Aminosäuren als Knoten, Bindungen als Kanten). LLMs wie ESMFold [16] werden zur Proteinstrukturvorhersage eingesetzt. Der Subgraph Algorithmus kann hier doppelt verwendet werden:

1. Zur Modellierung des LLM selbst als Graph.
2. Zur Analyse der vorhergesagten Proteinstrukturen als Subgraphen bekannter Protein-Familien.

Dies könnte die Laufzeit von Protein-Homologie-Suchen von $O(n^2)$ (Smith-Waterman [18]) auf $O(n^3)$ bei gleichzeitig höherer struktureller Präzision verbessern.

7.4.2 Quantencomputing: Quanten-Transformer

Quanten-LLMs [17] repräsentieren die Attention-Matrizen durch Quantenzustände. Der Quanten-Subgraph-Ansatz könnte die Laufzeit weiter auf $O(n^{1.5})$ reduzieren, analog zu Grover's Algorithmus für die Suche. Dies würde den Subgraph Algorithmus zu einem Quanten-Beschleuniger für LLM-Pattern-Matching machen.

7.4.3 Kryptographie: Sichere LLM-Inferenz

Homomorphe Verschlüsselung ermöglicht die Ausführung von Berechnungen auf verschlüsselten Daten. Der Subgraph Algorithmus kann auf verschlüsselte Attention-Matrizen angewendet werden: Da die Signaturfunktion eine reine arithmetische Operation ist, kann sie über verschlüsselten Daten ausgeführt werden (Signatur-Chiffre [4]). Dies würde datenschutzkonformes Pattern-Matching in LLMs ermöglichen.

7.4.4 Formale Sprachtheorie

Die Chomsky-Hierarchie klassifiziert formale Sprachen nach ihrer Ausdrucksstärke [15]. Transformer-LLMs können kontextuelle Abhängigkeiten beliebiger Reichweite modellieren (Typ-0-Sprachen). Der Subgraph-Ansatz ermöglicht die formale Klassifikation von Attention-Mustern nach der Chomsky-Hierarchie:

- Lokale Attention ($w = 1$) $\cong$ Reguläre Grammatik (Typ 3).
- Kausale Attention $\cong$ Kontextfreie Grammatik (Typ 2).
- Vollständige Attention $\cong$ Kontextsensitive Grammatik (Typ 1).

Dies verbindet die Theorie der Programmiersprachen mit der LLM-Forschung.

7.5 Offene Fragen und zukünftige Arbeiten

1. **Unbedingte untere Schranke:** Kann die $\Omega(n^3)$ -Schranke ohne die SETH-Annahme bewiesen werden?
2. **Kontinuierliche Attention-Gewichte:** Welche Erweiterung der Signaturfunktion ist optimal für reellwertige Attention-Matrizen?
3. **Dynamische Graphen:** Wie skaliert der Subgraph Algorithmus für Transformer, deren Attention-Graphen sich über die Zeit ändern (z. B. beim Fine-Tuning)?
4. **Cross-Modell-Transfer:** Können Attention-Pattern-Subgraphen als strukturelle Repräsentationen zwischen verschiedenen LLM-Architekturen transferiert werden?
5. **Theorie der LLM-Kapazität:** Gibt es eine formale Charakterisierung der maximalen Ausdruckskraft eines Transformers in Bezug auf die Anzahl der Subgraphen seiner Attention-Matrizen?
6. **Quanten-Subgraph Algorithmus:** Lässt sich der Subgraph Algorithmus auf Quantencomputern in $O(n^{1.5})$ implementieren?

8 Zusammenfassung

Diese Arbeit hat gezeigt, dass Transformer-Architekturen vollständig als gewichtete gerichtete Graphen formalisiert werden können, und dass das Erkennen struktureller Attention-Muster äquivalent zum Subgraph-Isomorphismus-Problem ist. Der zentrale Beitrag – der **Transformer-Subgraph-Satz** – beweist:

1. Jedes Attention-Pattern ist ein Subgraph des Transformer-Graphen G_T .
2. Der Subgraph Algorithmus erkennt diese Muster in $O(n^3)$ Zeit.
3. Die Subgraph-Hierarchie $G_{\text{local}} \subseteq G_{\text{causal}} \subseteq G_{\text{cross}} \subseteq G_{\text{full}} \subseteq G_T$ klassifiziert alle bekannten Attention-Mechanismen formal.
4. Unter SETH ist der Subgraph Algorithmus optimal für dieses Problem.

Die Ergebnisse verbinden die Graphentheorie, die Komplexitätstheorie und die LLM-Forschung auf eine formal präzise Weise und eröffnen zahlreiche neue Forschungsrichtungen.

Literatur

- [1] Epp, S. (2026). *Der Subgraph Algorithmus: Lösung des Subgraph-Isomorphismus-Problems in $O(n^3)$* . <https://github.com/hjstephan86/science>
- [2] Epp, S. (2026). *Formaler Beweis von $P = NP$ mittels Subgraph Algorithmus*. <https://github.com/hjstephan86/science>
- [3] Epp, S. (2026). *Universelle Signatur-Theorie: σ_j als domänenübergreifendes Isomorphie-Primitiv in 15 Disziplinen*. <https://github.com/hjstephan86/science>
- [4] Epp, S. (2026). *Signatur-Chiffre: Strukturbasierte Verschlüsselung auf Basis der injektiven Subgraph-Signaturfunktion*. <https://github.com/hjstephan86/science>
- [5] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention is all you need*. Advances in Neural Information Processing Systems, 30.
- [6] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of deep bidirectional transformers for language understanding*. NAACL-HLT 2019, 4171–4186.
- [7] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., et al. (2020). *Language models are few-shot learners*. Advances in Neural Information Processing Systems, 33, 1877–1901.
- [8] Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M., & Monfardini, G. (2009). *The graph neural network model*. IEEE Transactions on Neural Networks, 20(1), 61–80.
- [9] Abboud, A., Backurs, A., & Vassilevska Williams, V. (2015). *Tight hardness results for LCS and other sequence similarity measures*. FOCS 2015, 59–78.
- [10] Cordella, L. P., Foggia, P., Sansone, C., & Vento, M. (2004). *A (sub)graph isomorphism algorithm for matching large graphs*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 26(10), 1367–1372.
- [11] Ullmann, J. R. (1976). *An algorithm for subgraph isomorphism*. Journal of the ACM, 23(1), 31–42.
- [12] Voita, E., Talbot, D., Moiseev, F., Sennrich, R., & Titov, I. (2019). *Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned*. ACL 2019, 5797–5808.
- [13] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). *“Why should I trust you?”: Explaining the predictions of any classifier*. KDD 2016, 1135–1144.
- [14] Cook, S. A. (1971). *The complexity of theorem proving procedures*. STOC 1971, 151–158.
- [15] Chomsky, N. (1956). *Three models for the description of language*. IRE Transactions on Information Theory, 2(3), 113–124.

- [16] Lin, Z., Akin, H., Rao, R., Hie, B., Zhu, Z., Lu, W., et al. (2023). *Evolutionary-scale prediction of atomic-level protein structure with a language model*. Science, 379(6637), 1123–1130.
- [17] Gao, X., Duan, L., Gao, W., & Duan, Z. S. (2021). *Enhancing quantum machine learning with attention mechanism*. IEEE Transactions on Quantum Engineering, 2, 1–12.
- [18] Smith, T. F., & Waterman, M. S. (1981). *Identification of common molecular subsequences*. Journal of Molecular Biology, 147(1), 195–197.